10 · Security, Privacy & Compliance

> **Goal:** treat the ML system as an attack surface and a data-protection obligation: secure the endpoints, the training pipeline, the artifacts and the people.

■ Decision Tree: which component, and when?

■ How this module intersects the others

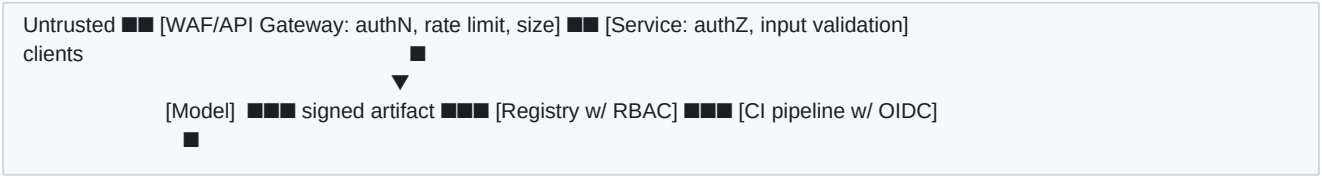
> Full course map: assets/tree_00_master_map.svg

Contents 1. ML threat model 2. Securing inference endpoints 3. Model poisoning & supply-chain attacks 4. Privacy: differential privacy & federated learning 5. RBAC & API gateway authentication 6. Edge cases & production failures 7. Senior interview questions & answers 8. Tips, Tricks & Field Notes 9. Worked Scenario: Suspected model extraction 10. More Interview Questions 11. Where This Module Connects 12. Interview Playbook: Tips & Tricks 13. Scenario-Based Evaluation 14. Rapid-Fire Round - 10.15 Privacy Engineering for ML Data: Minimisation, Retention and Erasure - 10.16 Regulation, Audit Readiness and Evidence Collection

---

1. ML threat model

Stage	Threat	Example	Primary controls
Data	**Data poisoning**	Attacker injects mislabelled samples via user feedback	Provenance, validation, outlier filters, robust training
Data	**Privacy leakage**	PII in training set	Minimisation, anonymisation, DP
Training	**Backdoor / trojan**	Trigger pattern → attacker-chosen output	Dataset audit, activation clustering, signed data
Artifact	**Malicious pickle / supply chain**	`pickle.load` runs arbitrary code	safetensors/ONNX, scanning, signing
Serving	**Evasion (adversarial examples)**	Small perturbation flips prediction	Input validation, adversarial training, rate-limits
Serving	**Model extraction**	Query API to clone model	Rate limit, quota, return labels not full probabilities, watermarking
Serving	**Membership inference / inversion**	Determine if a record was in training	DP, regularisation, restricted outputs
Serving	**DoS / resource exhaustion**	Huge payloads, expensive inputs	Size limits, timeouts, quotas, autoscale caps
Platform	**Credential theft, over-privileged jobs**	Training job with admin cloud role	Least privilege, workload identity, network policies
LLM-specific	**Prompt injection, data exfiltration**	Instructions hidden in retrieved docs	Content isolation, output filtering, tool permissions



```
                ▼ audit log (who, what model version, what input hash)
```

2. Securing inference endpoints

Layered: network → identity → request → model → output → audit.

```
# secure_api.py — authN (JWT), authZ (scopes), rate limit, size limit,
 output minimisation, audit
import hashlib, json, logging, os, time
from collections import defaultdict, deque

import jwt                              # PyJWT
from fastapi import Depends, FastAPI, HTTPException, Request
from fastapi.responses import JSONResponse
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from pydantic import BaseModel, ConfigDict, Field

JWT_PUBLIC_KEY_PEM = os.environ["JWT_PUBLIC_KEY"]    # PEM public key, injected via secret mana
ger, rotated (with a JWKS endpoint you would fetch keys by `kid`)
AUDIENCE, ISSUER = "fraud-api", "https://idp.example.com/"
MAX_BODY = 16_384
audit = logging.getLogger("audit")
app = FastAPI()
bearer = HTTPBearer(auto_error=True)

def model_score(features: list[float]) -> float:
    raise NotImplementedError("placeholder: call your loaded model here")

@app.middleware("http")
async def limit_body(request: Request, call_next):       # runs BEFORE the body is parsed into Req
    if int(request.headers.get("content-length") or 0) > MAX_BODY:
        return JSONResponse({"detail": "payload too large"}, status_code=413)
    return await call_next(request)                      # chunked bodies carry no length: also enforce a
t the gateway

class Req(BaseModel):
    model_config = ConfigDict(extra="forbid")
    features: list[float] = Field(min_length=3, max_length=3)

def authenticate(cred: HTTPAuthorizationCredentials = Depends(bearer)) -> dict:
    try:
        return jwt.decode(cred.credentials, JWT_PUBLIC_KEY_PEM, algorithms=["RS256"],   # never
 allow "none"/HS* confusion
                          audience=AUDIENCE, issuer=ISSUER, options={"require": ["exp"
;, "sub", "aud"]})
    except jwt.PyJWTError:
        raise HTTPException(401, "invalid token")

def require_scope(scope: str):
    def dep(claims: dict = Depends(authenticate)):
        if scope not in claims.get("scope", "").split():
            raise HTTPException(403, "insufficient scope")
        return claims
    return dep

_hits: dict[str, deque] = defaultdict(deque)
def rate_limit(sub: str, limit=100, window=60):          # per-replica only (effective limit = limit x re
plicas); in prod: Redis token bucket at the gateway
    now, q = time.time(), _hits[sub]
    while q and q[0] < now - window: q.popleft()
    if len(q) >= limit: raise HTTPException(429, "rate limit")
    q.append(now)
```

```
@app.post("/v1/predict")
async def predict(req: Req, request: Request, claims: dict = Depends(require_scope("predict:invoke&q
uot;))):
    rate_limit(claims["sub"])
    score = model_score(req.features)                    # your inference
    audit.info(json.dumps({"sub": claims["sub"], "model": os.getenv("M
ODEL_VERSION"),
        "input_sha256": hashlib.sha256(json.dumps(req.features).encode()).hexdigest(), "ts
": time.time()}))
    # output minimisation: bucketed decision instead of raw probability reduces extraction fidelity
    return {"decision": "review" if score > 0.8 else "allow"}
```

Hardening checklist - TLS everywhere; **mTLS** service-to-service (Istio PeerAuthentication: STRICT). - Don't expose raw logits/probabilities externally unless required; round or bucket. - Per-tenant quotas and anomaly detection on query patterns (extraction attempts show systematic grid-like queries). - Request size/time limits and complexity caps (e.g., max tokens, image dimensions). - Never log raw PII; log hashes or tokenised ids; separate access to logs. - Secrets from a manager (Vault, cloud KMS) via workload identity—never env files in images.

Istio mesh-level mTLS and authorization:

```
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata: { name: default, namespace: mlops }
spec: { mtls: { mode: STRICT } }
---
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata: { name: fraud-api-allow, namespace: mlops }
spec:
  selector: { matchLabels: { app: fraud-api } }
  action: ALLOW
  rules:
    - from: [{ source: { principals: ["cluster.local/ns/checkout/sa/checkout-svc"] } }]
      to: [{ operation: { methods: ["POST"], paths: ["/v1/predict"] } }]
```

3. Model poisoning & supply-chain attacks

Data poisoning defences | Control | Detail | |---|---| | Provenance | Only trusted, signed sources; track every dataset's origin | | Feedback-loop sanitation | User-generated labels pass through rate limits, reputation weighting, and review | | Statistical screening | Outlier/influence detection (e.g., loss-based filtering, spectral signatures) before training | | Golden set | Frozen, trusted evaluation set; poisoned retrain shows regression/backdoor tests fail | | Backdoor tests | Probe with candidate trigger patterns; compare with prior model on targeted slices | | Canary + shadow | Poisoned behaviours often appear only on targeted inputs: examine flip clusters |

Detecting suspicious training-data updates:

```
import numpy as np
from sklearn.ensemble import IsolationForest

def screen_new_batch(ref_X: np.ndarray, new_X: np.ndarray, max_flag_rate=0.05):
    iso = IsolationForest(contamination=0.01, random_state=0).fit(ref_X)
    flagged = iso.predict(new_X) == -1
    if flagged.mean() > max_flag_rate:
        raise RuntimeError(f"batch anomaly rate {flagged.mean():.1%} exceeds {max_flag_rate:.0%}; qu
arantine")
    return new_X[~flagged]
```

Artifact / supply-chain

```
# UNSAFE: arbitrary code execution on load
import pickle; pickle.load(open("model.pkl", "rb"))     # never for untrusted files
```

```
# SAFER: tensor-only formats
from safetensors.torch import load_file
state = load_file("model.safetensors")
```

Controls: sign model artifacts and verify at deploy (cosign/sigstore), hash pinning in manifests, restrict registry write access to CI identity, scan dependencies (pip-audit), private package mirror, hash-pinned requirements (pip install --require-hashes), SBOM.

```
cosign sign-blob --key cosign.key model.onnx --output-signature model.on
nx.sig
cosign verify-blob --key cosign.pub --signature model.onnx.sig model.onnx   # init container before servi
ng
```

4. Privacy: differential privacy & federated learning

Differential Privacy (DP)

A mechanism *M* is (ε, δ)-DP if adding/removing one individual changes output probabilities by at most e^ε (plus δ). Smaller ε → stronger privacy, lower utility.

Technique	Where	Trade-off
Laplace/Gaussian mechanism	Aggregate queries/statistics	Noise ∝ sensitivity/ε
DP-SGD	Model training: clip per-sample gradients, add Gaussian noise	Accuracy loss, 2–5× slower, needs large data
PATE	Teacher ensemble votes with noise	Needs public unlabeled data
Privacy budget accounting	Track cumulative ε across releases	Budget is finite

```
# DP-SGD with Opacus (PyTorch)
import torch
from opacus import PrivacyEngine

model = build_model(); opt = torch.optim.SGD(model.parameters(), lr=0.1)
pe = PrivacyEngine()
model, opt, loader = pe.make_private_with_epsilon(
    module=model, optimizer=opt, data_loader=train_loader,
    epochs=10, target_epsilon=3.0, target_delta=1e-5, max_grad_norm=1.0)
for epoch in range(10):
    for x, y in loader:
        opt.zero_grad(); loss_fn(model(x), y).backward(); opt.step()
    print(f"epoch {epoch} eps={pe.get_epsilon(delta=1e-5):.2f}")
```

Laplace mechanism for a released count:

```
import numpy as np
def dp_count(true_count: int, epsilon: float, sensitivity: float = 1.0) -> float:
    return true_count + np.random.laplace(0, sensitivity / epsilon)
```

Federated Learning (FL)

Train where the data lives; share model updates, not raw data.

```
sequenceDiagram
  participant S as Server
  participant C1 as Client A (hospital)
  participant C2 as Client B (hospital)
  S->>C1: global weights w_t
  S->>C2: global weights w_t
  C1->>C1: local training on private data
  C2->>C2: local training on private data
```

```
C1-->>S: update Δ1 (clipped, noised, secure-agg)
C2-->>S: update Δ2
S->>S: aggregate (FedAvg) -> w_{t+1}
```

FL is **not private by itself**: updates leak information → combine with secure aggregation and DP. Challenges: non-IID data, stragglers, communication cost, client poisoning (use robust aggregation: median/trimmed mean/Krum), debugging without data access.

```
# Minimal FedAvg aggregation (weighted by client sample counts)
import numpy as np
def fed_avg(updates: list[list[np.ndarray]], sizes: list[int]) -> list[np.ndarray]:
    total = sum(sizes)
    return [sum(u[i] * (n / total) for u, n in zip(updates, sizes)) for i in range(len(updates[0]))]
```

Frameworks: Flower, TensorFlow Federated, NVIDIA FLARE, PySyft.

Requirement	Pick
Release statistics/model publicly with formal guarantee	DP
Data cannot leave silos (regulation, contracts)	FL (+ secure aggregation)
Both, high-stakes	FL + DP-SGD at client + secure aggregation
Just reduce exposure	Minimisation, pseudonymisation, tokenisation, access control

5. RBAC & API gateway authentication

Kubernetes RBAC: least-privilege, namespaced, per-workload ServiceAccounts.

```
apiVersion: v1
kind: ServiceAccount
metadata: { name: fraud-trainer, namespace: mlops, annotations: { eks.amazonaws.com/role-arn: "arn:a
ws:iam::111122223333:role/fraud-trainer" } }
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: trainer, namespace: mlops }
rules:
  - { apiGroups: [""], resources: ["configmaps"], resourceNames: ["train-config&
quot;], verbs: ["get"] }
  - { apiGroups: ["batch"], resources: ["jobs"], verbs: ["create", "ge
t", "list"] }
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: trainer-binding, namespace: mlops }
subjects: [{ kind: ServiceAccount, name: fraud-trainer, namespace: mlops }]
roleRef: { kind: Role, name: trainer, apiGroup: rbac.authorization.k8s.io }
```

ML platform role matrix: | Role | Data (PII) | Experiments | Register model | Promote to prod | Prod serving config | Audit logs | |---|---|---|---|---|---|---| | Data scientist | masked only | R/W | ■ (staging) | ■ | ■ | | | ML engineer | masked | R/W | ■ | ■ w/ approval | R | ■ | | Approver / Risk | ■ | R | ■ | ■ (approve) | ■ | R | | CI service principal | scoped | W | ■ | ■ (only identity that can write alias) | ■ | W | | SRE / on-call | ■ | ■ | ■ | rollback only | R/W | R | | Auditor | ■ | R | R | R | R | R |

API gateway (Kong example): JWT/OIDC auth, rate limiting, request size, IP restrictions — enforced before traffic reaches the model.

```
_format_version: "3.0"
services:
  - name: fraud-api
    url: http://fraud-api.mlops.svc.cluster.local:80
    routes:
```

```
    - { name: predict, paths: [/v1/predict], methods: [POST] }
  plugins:
    - name: openid-connect
      config: { issuer: https://idp.example.com/.well-known/openid-configuration, auth_methods: [bearer
], scopes_claim: [scope], scopes_required: [predict:invoke] }
    - { name: rate-limiting, config: { minute: 600, policy: redis, redis_host: redis, limit_by: consume
r } }
    - { name: request-size-limiting, config: { allowed_payload_size: 16, size_unit: kilobytes } }
    - { name: ip-restriction, config: { allow: ["10.0.0.0/8"] } }
```

Licence note: the `openid-connect` plugin is, to my knowledge, part of Kong Gateway Enterprise rather than the open-source edition; with OSS Kong use the jwt plugin (or another OSS auth plugin) instead, and verify plugin availability against current Kong documentation.

Authentication choices: **API keys** (simple, per-client, rotate; weak identity), **OAuth2/OIDC JWT** (short-lived, scopes; default for user/service), **mTLS** (service identity; mesh), **workload identity** (cloud IAM, no static secrets).

6. Edge cases & production failures

Failure	Root cause	Fix
Model extracted via public API	Unlimited queries returning full probabilities	Rate limit, per-tenant quota, bucketed outputs, monitor query diversity
PII in prediction logs	Debug logging of full payloads	Redact/tokenise at log boundary; log hashes; retention limits
Poisoned feedback labels	Users/bots rate-manipulate "thumbs"	Weighted by reputation, review sample, anomaly checks, limit per-user influence
Over-privileged training job	Shared cluster-admin ServiceAccount	Per-job SA, least privilege, workload identity, network policy
JWT algorithm confusion	Accepting HS256 with RS256 public key / `alg=none`	Explicit allow-list of algorithms; validate `aud`/`iss`/`exp`
Leaky error messages	Stack traces exposing paths/versions	Generic client errors, detailed server logs
Secrets in container image	`COPY .env`	`.dockerignore`, secret manager, image scanning for secrets
DP "applied" with huge ε	Marketing ε=50	Report ε, δ, accounting method; set policy ceiling; evaluate with membership inference tests
Federated client poisoning	One malicious participant scales its update	Norm clipping, robust aggregation, anomaly detection
Compliance gap on deletion	Subject data persists in feature store/snapshots	Lineage-driven erasure, retraining schedule, documentation

7. Senior interview questions & answers

Q1. How would you protect a model API from being cloned? - ■ *Trap:* "Put it behind authentication." - ■ *Staff:* Authentication is necessary but not sufficient—paying customers can still extract. Add per-identity rate limits and quotas, return minimum information (labels/buckets instead of probabilities), monitor for systematic query patterns (grid sampling, high input diversity), add watermarking/canary outputs, legal terms, and cost/benefit awareness: perfect protection is impossible, aim to make extraction uneconomical and detectable.

Q2. What is data poisoning and how would you defend a continuously-trained model? - ■ *Trap:* "Clean the data before training." - ■ *Staff:* Poisoning inserts crafted samples to degrade accuracy or implant a backdoor. Defence in depth: trusted provenance and signing, limits on any single source's influence, anomaly screening before training, training on snapshots, golden-set and backdoor probe evaluation, champion/challenger comparison focused on targeted slices, human approval for

high-risk models, and retaining snapshots for rollback and forensics.

Q3. Differential privacy vs federated learning—when each? - ■ *Trap:* "They're the same: both protect privacy." - ■ *Staff:* FL addresses *where data lives* (no centralisation), DP addresses *what can be inferred from outputs* (formal bound). FL alone leaks via gradients; DP alone requires centralised data. For siloed sensitive data I'd use FL with secure aggregation and DP noise; for centrally-held data where release risk is the concern, DP-SGD or DP statistics. Set ε through risk assessment and measure utility loss.

Q4. Why is pickle dangerous for model artifacts and what do you do instead? - ■ *Trap:* "It's just a serialization format." - ■ *Staff:* Unpickling executes arbitrary callables, so loading an untrusted or tampered artifact is remote code execution inside your serving environment. Use tensor-only formats (safetensors, ONNX), verify signatures/hashes before load, restrict registry write access, and load in sandboxed, least-privilege pods with no outbound network.

Q5. Design authN/authZ for an internal model platform with many teams. - ■ *Trap:* "Shared API key per team." - ■ *Staff:* OIDC SSO for humans with group-based RBAC; workload identity (cloud IAM roles bound to ServiceAccounts) for pipelines, no long-lived keys; service-to-service mTLS with authorisation policies; registry permissions per namespace; separate promote vs train rights; every privileged action audited to immutable storage. Gateway enforces tokens, scopes, quotas.

Q6. What do you log for audit without violating privacy? - ■ *Trap:* "Log everything including request bodies." - ■ *Staff:* Log metadata sufficient to reconstruct: request id, caller identity, model name/version, timestamp, decision, a hash or tokenised reference to inputs (with the raw features available in a restricted store under retention policy), and reason codes. Protect logs with access control, encryption and retention; allow deletion mapping for erasure.

Q7. How do you handle a GDPR "right to be forgotten" request for a deployed model? - ■ *Trap:* "We delete the user from the database." - ■ *Staff:* Use lineage to remove the subject from raw stores, feature tables, caches and training snapshots; flag for exclusion in future training; decide, with legal, whether current model retraining is required (and by when) — typically at next scheduled retrain, or immediately if the model memorises records (assess via membership inference). Document evidence. DP training offers formal reasoning that no single record dominates.

Q8. An adversary crafts inputs to evade your fraud model. Mitigations? - ■ *Trap:* "Retrain more often." - ■ *Staff:* Treat it as an arms race: don't expose scores, add randomised/ensemble defenses and non-model rules, adversarial training with generated perturbations, monitor for distribution shift in rejected/accepted patterns, fast feedback from confirmed fraud, rate-limit probing accounts, and use features hard to manipulate (behavioural/graph) over trivially-controllable ones.

8. Tips, Tricks & Field Notes

1. Never `pickle.load` untrusted files; use safetensors or ONNX and verify signatures.
2. Allow-list JWT algorithms; always validate aud, iss and exp.
3. Return decisions or buckets, not raw probabilities, on public APIs.
4. Per-tenant quotas plus anomaly detection catch extraction attempts.
5. Log hashes or tokens instead of raw PII.
6. Split train and promote rights; only CI can write the champion alias.

9. Worked Scenario: Suspected model extraction

Situation. One API key issues grid-like queries at 3 AM.

Steps

1. Rate-limit and flag the key.
2. Check query diversity and systematic patterns.
3. Switch outputs to buckets for that tenant.
4. Review contract terms and add watermark canaries.

Outcome. Key revoked; per-tenant quotas and bucketed outputs become defaults.

10. More Interview Questions

Q9. DP vs federated learning? - ■ *Trap:* Same thing. - ■ *Staff:* FL is about where data lives; DP bounds what outputs reveal. Combine FL, secure aggregation and DP for high-stakes siloed data.

Q10. Defend a continuously-trained model from poisoning? - ■ *Trap:* Clean the data. - ■ *Staff:* Provenance, per-source influence limits, anomaly screening, golden-set and backdoor tests, snapshots for rollback.

Q11. GDPR erasure for a deployed model? - ■ *Trap:* Delete the DB row. - ■ *Staff:* Lineage to all stores and snapshots, exclude from next training, assess memorisation, document evidence.

11. Where This Module Connects

Direction	Module	What flows
■ Fed by	04 Registry	RBAC + approvals
■ Feeds	06 Serving	endpoint hardening
■ Feeds	09 Infra & K8s	pod + supply-chain security

12. Interview Playbook: Tips & Tricks

1. Frame answers with a threat model: asset, adversary, control.
2. Always mention rate limits and output minimisation for model extraction.
3. Use 'least privilege' and 'workload identity' for platform questions.
4. Describe DP and FL as complementary, not alternatives.
5. Mention pickle risk and signed artifacts.
6. Mention log redaction and retention for privacy.

13. Scenario-Based Evaluation

Each scenario shows what the interviewer is really testing, the answer that loses points, and the answer that earns them.

Scenario 1: Extraction in progress

Situation. A competitor's account queries your API systematically.

What is being evaluated. Detection and response.

- ■ **Weak answer:** Block them.
- ■ **Strong answer:** 1. Monitor query diversity and patterns per key. 2. Throttle, then bucket outputs. 3. Preserve logs for legal action. 4. Add watermark canary outputs.

Likely follow-up: *How do you keep paying customers happy?*

Scenario 2: Poisoned feedback

Situation. A bot farm manipulates thumbs-up feedback used for training.

What is being evaluated. Data integrity.

- ■ **Weak answer:** Ignore noisy labels.
- ■ **Strong answer:** 1. Limit per-user influence and use reputation weighting. 2. Anomaly detection on feedback patterns. 3. Golden-set regression checks. 4. Human review sample before training.

Likely follow-up: *How do you recover a model already trained?*

Scenario 3: Leaked token

Situation. An API key appears in a public repo.

What is being evaluated. Incident response.

- ■ **Weak answer:** Remove from repo.
- ■ **Strong answer:** 1. Revoke and rotate immediately. 2. Audit usage during exposure. 3. Secret scanning in CI and pre-commit. 4. Move to workload identity.

Likely follow-up: *How to prevent recurrence?*

Scenario 4: Erasure request

Situation. User requests deletion; data is in features, snapshots and logs.

What is being evaluated. Compliance operations.

- ■ **Weak answer:** Delete DB row.
- ■ **Strong answer:** 1. Lineage to find all copies. 2. Delete/tokenise and record evidence. 3. Exclude from future training. 4. Assess memorisation and retrain timeline.

Likely follow-up: *How do you prove it?*

14. Rapid-Fire Round

Question	One-line answer
Pickle risk?	Arbitrary code execution on load.
JWT alg confusion?	Accepting none/HS with public key.
DP epsilon?	Privacy loss bound.
FL leak?	Updates reveal data; add secure aggregation.
mTLS purpose?	Service identity and encryption.
RBAC principle?	Least privilege.
Membership inference?	Detect if record was in training.
Why bucket outputs?	Reduce extraction fidelity.
## 10.15 Privacy Engineering for ML Data: Minimisation, Retention and Erasure	

Conceptual Deep Dive

Start with a **data inventory**: every table, feature, log stream, snapshot and model, classified as PII, sensitive attribute, derived data or non-personal, with an owner and a stated purpose. GDPR-style concepts (purpose limitation, data minimisation, storage limitation, a lawful basis for processing) are the vocabulary, but this is orientation, not legal advice; verify current text with counsel. Without an inventory you cannot answer "where does this user's data live".

Minimisation in rough order of strength: do not collect; drop fields; coarsen (age bands, truncated ZIP); aggregate; tokenise; pseudonymise. **Pseudonymisation** with a keyed hash (HMAC) gives stable join keys without exposing identifiers, but it remains personal data because the key holder can re-link, and an unkeyed hash of a low-entropy value (email, phone) falls to a dictionary attack. **Anonymisation** is hard: k-anonymity fails against linkage attacks and homogeneity within a group, so treat "anonymous" as a claim that needs a re-identification test.

Retention differs per class: raw events, feature tables, point-in-time snapshots, prediction logs, training sets and models each get a retention period tied to purpose and risk tier (4.15). Short-lived raw data with longer-lived aggregates is a common pattern.

Erasure is lineage-driven: from a subject id find source rows, then every downstream feature, snapshot, training set and model derived from them. Rows can be deleted from stores; models cannot easily forget. Options: retrain on a schedule that bounds residual exposure, exact unlearning by retraining or sharded training (expensive), or approximate unlearning (cheap, weaker guarantees). Document the residual risk and the schedule. Backups need an expiry policy or a deletion log replayed on restore.

Privacy in logging: redact at the log boundary, restrict access, never log raw payloads for debugging, and compute drift metrics on aggregates or hashed values. **Privacy testing**: membership-inference audits, canary records planted in training data, and DP accounting where differential privacy is used (see Module 10 section 4 (legacy)).

Edge cases: an erased user persists in an old snapshot used to train a live model; unkeyed hashes reversed by guessing; backups restoring erased data.

Cross-Connection Scenarios

- Lineage metadata comes from chapter 2 and registry records in 4.16.

- Retention and evidence interplay with audit logs in 10.16.
- Prediction logging and monitoring boundaries are in 8.18.
- Fairness audits that need sensitive attributes are in 17.1.

Production Anti-Patterns

- Logging raw payloads for debugging: PII sprawls into systems with weak access control and long retention.
- Treating pseudonymised data as anonymous: it stays regulated and is re-linkable.
- No inventory of where user data flows: erasure requests cannot be completed or proven.

Minimal Working Code

An HMAC pseudonymiser and a log redaction filter, with checks including the unkeyed-hash weakness.

```
import hashlib, hmac, logging, re

def pseudonymise(value: str, key: bytes) -> str:
    return hmac.new(key, value.strip().lower().encode(), hashlib.sha256).hexdigest()[:16]

EMAIL = re.compile(r"[\w.+-]+@[\w-]+\.[\w.-]+")
PHONE = re.compile(r"(?<!\w)\+?\d[\d\s().-]{7,}\d")

def redact(msg: str) -> str:
    return PHONE.sub("[PHONE]", EMAIL.sub("[EMAIL]", msg))

class RedactFilter(logging.Filter):
    def filter(self, record):
        record.msg, record.args = redact(record.getMessage()), ()
        return True

k1, k2 = b"key-one", b"key-two"
assert pseudonymise("A@x.com ", k1) == pseudonymise("a@x.com", k1)   # stable within
a key
assert pseudonymise("a@x.com", k1) != pseudonymise("a@x.com", k2)    # differs across
 keys
plain = hashlib.sha256(b"a@x.com").hexdigest()                       # unkeyed hash
guesses = ["b@x.com", "a@x.com"]
print("dictionary attack recovers:", [g for g in guesses if hashlib.sha256(g.encode()).hexdiges
t() == plain])
rec = logging.LogRecord("t", logging.INFO, "", 0, "user a@x.com called +1 415 55
5 0100", (), None)
RedactFilter().filter(rec)
assert "@" not in rec.msg and "415" not in rec.msg
print(rec.msg)
```

A lineage-driven erasure planner.

```
from collections import deque

def downstream(graph: dict, roots: set) -> set:
    seen, q = set(roots), deque(roots)
    while q:
        for child in graph.get(q.popleft(), []):
            if child not in seen:
                seen.add(child)
                q.append(child)
    return seen

def erasure_plan(graph, kinds, holders, subjects):
    """holders: source dataset -> set of subject ids it contains."""
    roots = {d for d, subs in holders.items() if subs & set(subjects)}
    plan = {"delete_rows": [], "recompute": [], "retrain_or_rotate": []}
    for node in sorted(downstream(graph, roots)):
        kind = kinds[node]
```

```
        key = {"dataset": "delete_rows", "feature": "recompute",
"model": "retrain_or_rotate"}[kind]
        plan[key].append(node)
    return plan

graph = {"events_raw": ["features_v3", "train_2025q4"], "features_v3&q
uot;: ["train_2025q4"],
         "train_2025q4": ["churn_model_v7"], "crm_dump": []}
kinds = {"events_raw": "dataset", "features_v3": "feature", "
;train_2025q4": "dataset",
         "churn_model_v7": "model", "crm_dump": "dataset"}
holders = {"events_raw": {"u1", "u2"}, "crm_dump": {"u9"
;}}
plan = erasure_plan(graph, kinds, holders, ["u1"])
assert "churn_model_v7" in plan["retrain_or_rotate"] and "crm_dump" not in
plan["delete_rows"]
print(plan)
```

Interview Angle

Q: A user requests deletion. Trace what must be erased or retrained in an ML system. **A:** Use the inventory and lineage to find the source rows, then every derived feature table, snapshot, training set, prediction log and backup. Delete or recompute those; flag models trained on them for the next scheduled retrain or an immediate one if risk demands. Trade-off: immediate retraining is costly, a bounded schedule leaves residual exposure that must be documented. Check: an automated report listing each store cleaned and the model retrain date, within the legal deadline.

Q: Why isn't hashing an email address sufficient anonymisation? **A:** Emails are low-entropy and guessable, so an attacker hashes candidates and matches; the hash is also a stable identifier that links records across datasets. A keyed HMAC defeats offline guessing without the key but is still pseudonymous personal data because the key holder can re-link. Real anonymisation needs aggregation or generalisation plus a re-identification risk test. Check: attempt a linkage attack with an auxiliary dataset and measure the match rate.

10.16 Regulation, Audit Readiness and Evidence Collection

Conceptual Deep Dive

At concept level, three regimes matter to ML platforms: **data protection** (GDPR-style: lawful basis, minimisation, subject rights, automated-decision provisions), **sector model-risk rules** (banking, insurance, healthcare supervision), and **AI-specific regulation** (the EU AI Act uses a risk-tier concept with stronger duties for high-risk uses). Details, dates and scope change; this section is orientation, not legal advice, so verify against primary sources and assign someone to track change.

The engineering pattern is a **controls map**: each legal or policy requirement becomes a technical control and a named **evidence source**. Example: "decisions must be reproducible" maps to the control "every prediction logs model version and feature snapshot id" with evidence "prediction log query" and "registry record". A requirement with no automated evidence is a gap, however well documented. Controls documented but not enforced are the commonest audit failure, so enforce in CI and protected environments (4.15), and test the control.

Evidence by design: immutable (append-only or hash-chained) audit logs, versioned model cards, decision records, access logs and change history, all retrievable by model version and request id. Evidence that exists but cannot be retrieved within the audit window is functionally absent, so run **audit drills**: pick one prediction from months ago and assemble its pack (inputs or feature snapshot, model version and approval, explanation reasons, human-review outcome) under a time limit.

Explainability and contestability: automated decisions with significant effects may need meaningful explanations, a route to human review and a way to contest; build reason codes (17.3) and a human-in-the-loop path (17.4). **Third-party and vendor models**: due diligence, data processing agreements, change notification, evaluation on your own data and an exit plan. Edge cases: evidence that is slow to retrieve; controls not actually enforced; a vendor model that changes silently behind a stable endpoint (pin versions, run canary evals).

Cross-Connection Scenarios

- Risk tiers and approvals feed the controls map (4.15).
- Erasure and retention evidence connect to 10.15.

- Reproducibility fields come from prediction logging in 8.18 and the registry in 4.16.
- Governance artifacts and review boards are in 17.4.

Production Anti-Patterns

- Compliance as a one-time document: the system drifts away from it within a quarter.
- Evidence in ad-hoc spreadsheets: not tamper-evident, not queryable, not tied to model versions.
- No owner for regulatory change tracking: new obligations are discovered during an audit.

Minimal Working Code

A control-to-evidence checker that reports gaps in a sample metadata store.

```
CONTROLS = {
    "C1 reproducible decisions": [("prediction_log", "model_version"), (&qu
ot;prediction_log", "feature_snapshot_id")],
    "C2 independent approval": [("registry", "approver"), ("registry&q
uot;, "validation_report")],
    "C3 contestability": [("registry", "reason_code_spec"), ("process&
quot;, "appeal_runbook")],
}

def check_controls(controls: dict, store: dict, model: str) -> dict:
    gaps = {}
    for name, queries in controls.items():
        missing = [f"{src}.{field}" for src, field in queries
                   if not store.get(model, {}).get(src, {}).get(field)]
        if missing:
            gaps[name] = missing
    return gaps

store = {"credit_v7": {
    "prediction_log": {"model_version": "7", "feature_snapshot_id"
:: "snap-123"},
    "registry": {"approver": "raj", "validation_report": "&q
uot;},
    "process": {"appeal_runbook": "wiki/appeals"}}}
gaps = check_controls(CONTROLS, store, "credit_v7")
for control, missing in gaps.items():
    print("GAP", control, "->", missing)
assert set(gaps) == {"C2 independent approval", "C3 contestability"}
```

An HMAC-signed decision-record bundle and verifier.

```
import hashlib, hmac, json

def build_bundle(request_id, store, key: bytes) -> dict:
    body = {"request_id": request_id, "model_version": store["model_version"
;],
            "feature_snapshot": store["feature_snapshot"], "reasons": store
["reasons"],
            "approvals": store["approvals"]}
    payload = json.dumps(body, sort_keys=True).encode()
    return {"body": body, "sig": hmac.new(key, payload, hashlib.sha256).hexdigest()}

def verify_bundle(bundle: dict, key: bytes) -> bool:
    payload = json.dumps(bundle["body"], sort_keys=True).encode()
    expected = hmac.new(key, payload, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, bundle["sig"])

key = b"audit-signing-key"          # in practice from a KMS, never in code
store = {"model_version": "credit:7", "feature_snapshot": {"income_ban
d": "B", "dti": 0.31},
         "reasons": ["high_dti", "short_history"], "approvals": [
```

```
"raj@2026-03-02"]}
bundle = build_bundle("req-001", store, key)
assert verify_bundle(bundle, key)
bundle["body"]["reasons"] = ["none"]      # tamper
print("after tamper, valid:", verify_bundle(bundle, key))
```

Interview Angle

Q: A regulator asks you to explain a decision from six months ago. What do you produce and what must exist beforehand? **A:** An evidence pack by request id: model version and its approval record, the feature snapshot or inputs, the explanation reason codes, thresholds, any human review outcome, and the model card of that version. Beforehand: prediction logging with version and snapshot ids, immutable audit logs, retained registry artifacts. Trade-off: storing inputs conflicts with minimisation, so store minimal snapshots under retention rules. Check: a timed drill assembles the pack within the agreed window.

Q: How do you make an ML platform audit-ready by default? **A:** Make the compliant path the only path: promotion gates that require evidence fields, append-only logs, model cards generated from registry metadata, and a controls-to-evidence matrix checked in CI and on a schedule. Add periodic audit drills and a named owner for regulatory change. Trade-off: platform friction for teams. Check: percentage of controls with automated evidence, and drill retrieval time.

Prev: ← 09 · **Next:** 11 · Production Failures & Troubleshooting →